



Lab-4

Title: Disk Scheduling Algorithm

Date:2080/12/30

Disk Scheduling Algorithm:

A Process makes the I/O requests to the operating system to access the disk. Disk Scheduling Algorithm manages those requests and decides the order of the disk access given to the requests. Disk Scheduling Algorithms are needed because a process can make multiple I/O requests and multiple processes run at the same time. The requests made by a process may be located at different sectors on different tracks. Due to this, the seek time may increase more. These algorithms help in minimizing the seek time by ordering the requests made by the processes.

Important Terms related to Disk Scheduling Algorithms

Seek Time - It is the time taken by the disk arm to locate the desired track.

Rotational Latency - The time taken by a desired sector of the disk to rotate itself to the position where it can access the Read/Write heads is called Rotational Latency.

Transfer Time - It is the time taken to transfer the data requested by the processes.

Disk Access Time - Disk Access time is the sum of the Seek Time, Rotational Latency, and Transfer Time.

Tools: Eclipse

Language: JAVA

Lab 4.1: First Come First Serve (FCFS)

First Come First Serve (FCFS)

In this algorithm, the requests are served in the order they come. Those who come first are served first. This is the simplest algorithm.

Disadvantages

- The scheme does not optimize the seek time.
- The request may come from different processes therefore there is the possibility of inappropriate movement of the head.

Source Code:

```
package operatingSystemLab;
import java.util.Scanner;
public class FcfsDisk {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the number of cylinders: ");
        int cylinders = scanner.nextInt();
        System.out.print("Enter the number of requests: ");
        int requests = scanner.nextInt();
        int[] diskQueue = new int[requests];
        System.out.println("Enter the disk requests:");
        for (int i = 0; i < requests; i++) {
            diskQueue[i] = scanner.nextInt();
        }
        System.out.print("Enter the current disk arm position: ");
        int head = scanner.nextInt();
```

```

int totalSeekTime = calculateSeekTime(diskQueue, head);
System.out.println("Total seek time using FCFS: " + totalSeekTime);
}

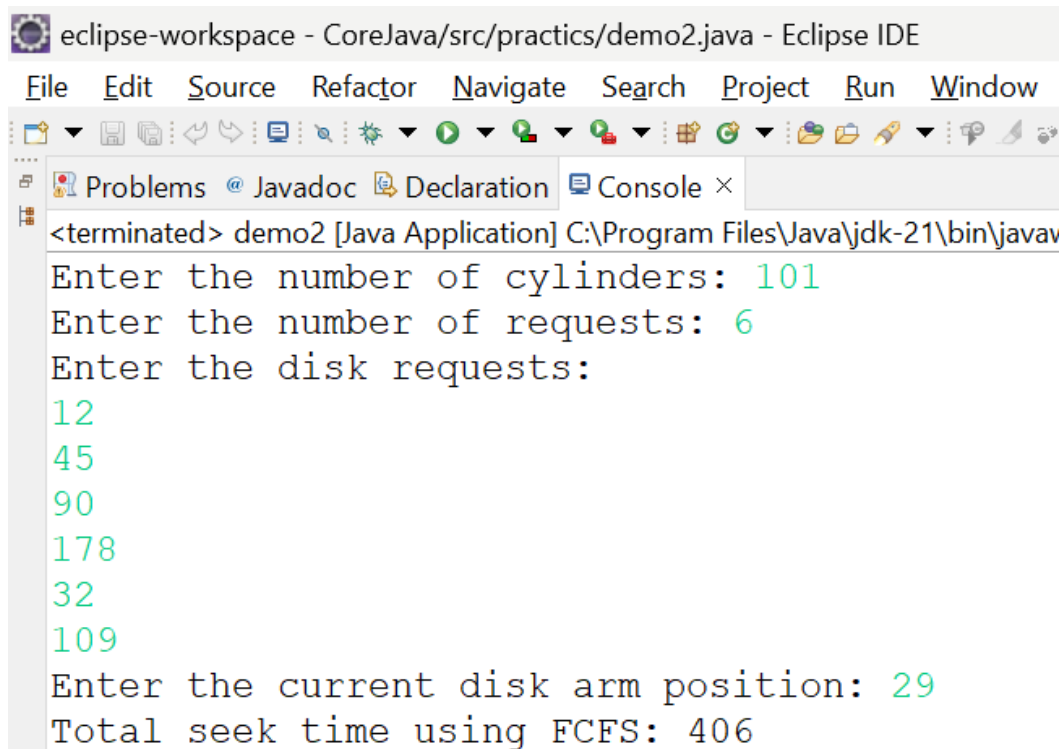
public static int calculateSeekTime(int[] diskQueue, int head) {
    int totalSeekTime = 0;

    // Iterate over disk queue and calculate seek time
    for (int i = 0; i < diskQueue.length; i++) {
        totalSeekTime += Math.abs(head - diskQueue[i]);

        // Move head to current request position
        head = diskQueue[i];
    }
    return totalSeekTime;
}
}

```

Output:



The screenshot shows the Eclipse IDE interface with the console window open. The title bar reads "eclipse-workspace - CoreJava/src/practics/demo2.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, and Window. The toolbar contains various icons for file operations, running, and debugging. The console window shows the following output:

```

<terminated> demo2 [Java Application] C:\Program Files\Java\jdk-21\bin\javav
Enter the number of cylinders: 101
Enter the number of requests: 6
Enter the disk requests:
12
45
90
178
32
109
Enter the current disk arm position: 29
Total seek time using FCFS: 406

```

Lab 4.2: Shortest Seek Time First(SSTF):

Shortest Seek Time First(SSTF):

Shortest seek time first (SSTF) algorithm selects the disk I/O request which requires the least disk arm movement from its current position regardless of the direction. It reduces the total seek time as compared to FCFS.

It allows the head to move to the closest track in the service queue.

Disadvantages

- It may cause starvation for some requests.
- Switching direction on the frequent basis slows the working of algorithm.
- It is not the most optimal algorithm.

Source code:

```
package operatingSystemLab;

import java.util.Arrays;
import java.util.Comparator;
import java.util.Scanner;

class SSTF {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the number of cylinders: ");

        int cylinders = scanner.nextInt();

        System.out.print("Enter the number of requests: ");

        int requests = scanner.nextInt();

        int[] diskQueue = new int[requests];

        System.out.println("Enter the disk requests:");

        for (int i = 0; i < requests; i++) {

            diskQueue[i] = scanner.nextInt();

        }

    }

}
```

```

    }

    System.out.print("Enter the current disk arm position: ");

    int head = scanner.nextInt(); scanner.close();

    // Calculate total seek time using SSTF algorithm
    int totalSeekTime = calculateSeekTime(diskQueue, head);

    System.out.println("Total seek time using SSTF: " + totalSeekTime);

    }

    // Function to calculate total seek time using SSTF algorithm
    public static int calculateSeekTime(int[] diskQueue, int head) {
        int totalSeekTime = 0;

        boolean[] visited = new boolean[diskQueue.length];

        // to track visited requests
        Arrays.fill(visited, false);

        for (int i = 0; i < diskQueue.length; i++) {
            int minDistance = Integer.MAX_VALUE;
            int minIndex = -1;

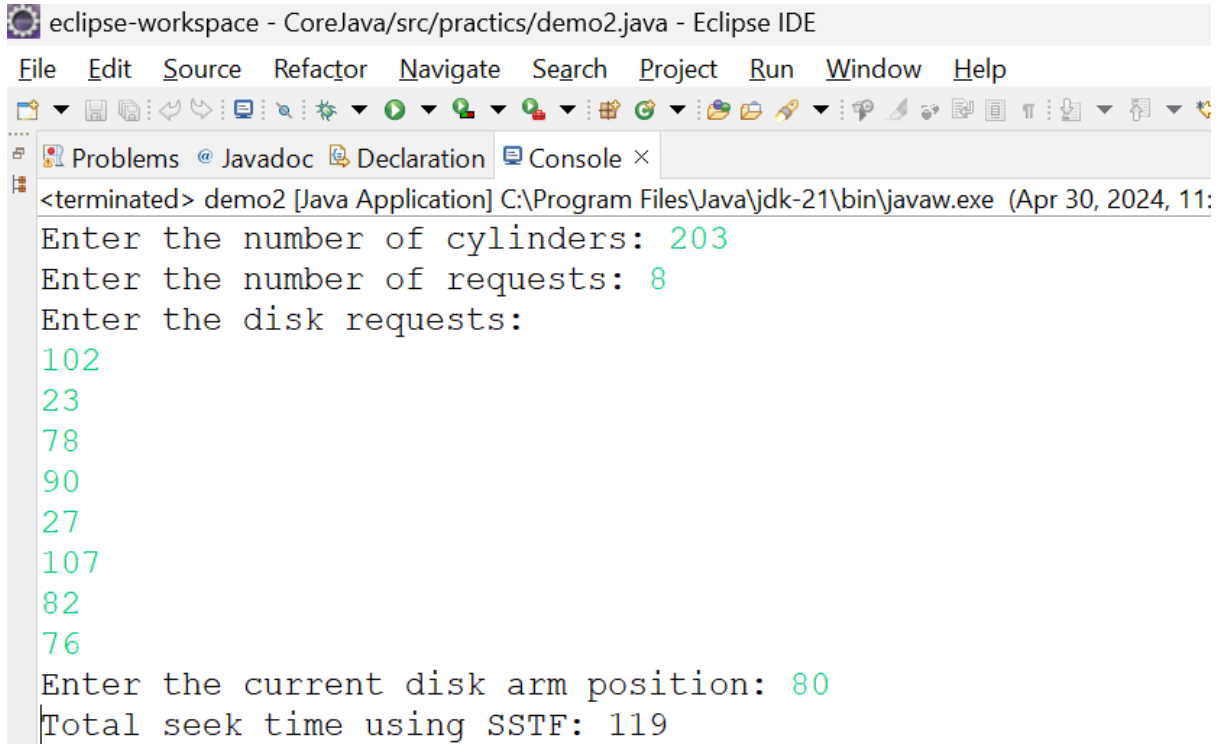
            for (int j = 0; j < diskQueue.length; j++) {
                if (!visited[j]) {
                    int distance = Math.abs(diskQueue[j] - head);
                    if (distance < minDistance) {
                        minDistance = distance;
                        minIndex = j;
                    }
                }
            }

            visited[minIndex] = true;
            totalSeekTime += minDistance;
            head = diskQueue[minIndex];
        }

        return totalSeekTime;
    }
}

```

Output:



The screenshot shows the Eclipse IDE interface. The title bar reads "eclipse-workspace - CoreJava/src/practics/demo2.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations and development tools. The "Console" tab is active, displaying the following output:

```
<terminated> demo2 [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Apr 30, 2024, 11:
Enter the number of cylinders: 203
Enter the number of requests: 8
Enter the disk requests:
102
23
78
90
27
107
82
76
Enter the current disk arm position: 80
Total seek time using SSTF: 119
```

Lab 4.3: Scan Algorithm

Scan Algorithm:

It is also called as Elevator Algorithm. In this algorithm, the disk arm moves into a particular direction till the end, satisfying all the requests coming in its path, and then it turns back and moves in the reverse direction satisfying requests coming in its path.

It works in the way an elevator works, elevator moves in a direction completely till the last floor of that direction and then turns back.

Source code:

```
package operatingSystemLab;

import java.util.Arrays;
import java.util.Scanner;

public class ScanDiskScheduling {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the number of cylinders: ");

        int cylinders = scanner.nextInt();

        System.out.print("Enter the number of requests: ");

        int requests = scanner.nextInt();

        int[] diskQueue = new int[requests];

        System.out.println("Enter the disk requests:");

        for (int i = 0; i < requests; i++) {

            diskQueue[i] = scanner.nextInt();

        }

        System.out.print("Enter the current disk arm position: ");

        int head = scanner.nextInt();

        scanner.close();

    }

}
```

```

        // Calculate total seek time using SCAN algorithm

        int totalSeekTime = calculateSeekTimeSCAN(diskQueue, head, cylinders);

        System.out.println("Total seek time using SCAN: " + totalSeekTime);

    }

    // Function to calculate total seek time using SCAN algorithm
    public static int calculateSeekTimeSCAN(int[] diskQueue, int head, int cylinders) {

        int totalSeekTime = 0;

        Arrays.sort(diskQueue);    // Sort the disk queue

        // Find the index of the head in the sorted disk queue

        int headIndex = Arrays.binarySearch(diskQueue, head);

        // If head is not found, insert it at the appropriate position
        if (headIndex < 0) {

            headIndex = -(headIndex + 1);

            int[] newDiskQueue = new int[diskQueue.length + 1];

            System.arraycopy(diskQueue, 0, newDiskQueue, 0, headIndex);

            newDiskQueue[headIndex] = head;

            System.arraycopy(diskQueue, headIndex, newDiskQueue, headIndex + 1, diskQueue.length
                - headIndex);

            diskQueue = newDiskQueue;

        }

        // Scan towards the right

        for (int i = headIndex; i < diskQueue.length; i++) {

            totalSeekTime += Math.abs(head - diskQueue[i]);

            head = diskQueue[i];

        }

        // Scan towards the left
        for (int i = headIndex - 1; i >= 0; i--) {

            totalSeekTime += Math.abs(head - diskQueue[i]);

            head = diskQueue[i];

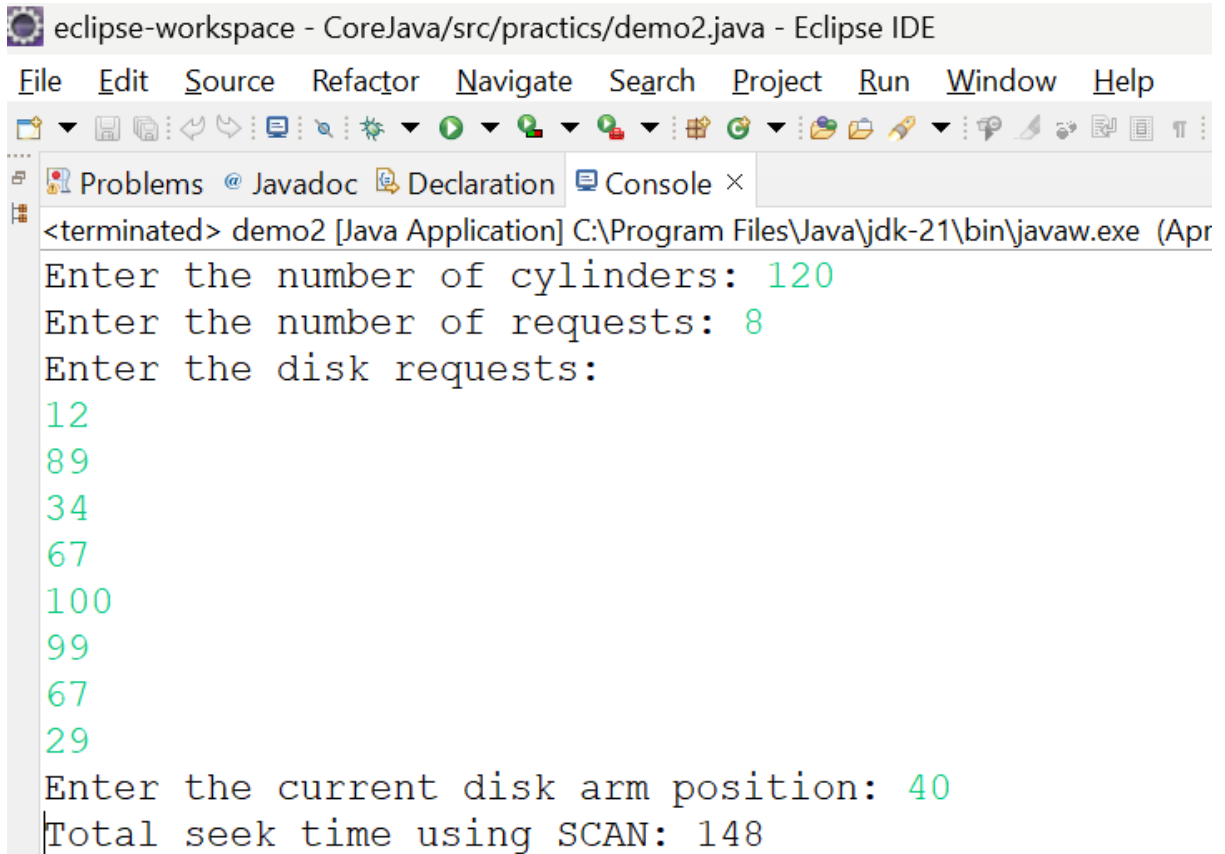
        }

        return totalSeekTime;

    }
}

```

Output:



The screenshot shows the Eclipse IDE interface. The title bar reads "eclipse-workspace - CoreJava/src/practics/demo2.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations, debugging, and development. The "Console" tab is active, showing the following output:

```
<terminated> demo2 [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Apr
Enter the number of cylinders: 120
Enter the number of requests: 8
Enter the disk requests:
12
89
34
67
100
99
67
29
Enter the current disk arm position: 40
Total seek time using SCAN: 148
```